

Projet de Fin d'Études

Système de Gestion d'une Coopérative Laitière

Réalisé par : [Votre Nom]

Encadré par : [Nom de l'encadrant]

Année académique : 2024/2025

Résumé

Ce projet a pour objectif la conception et le développement d'une application de gestion pour une coopérative laitière. Cette solution facilite l'enregistrement des adhérents, la gestion des livraisons de lait, la facturation et le suivi des paiements. L'application est développée avec Lazarus (Free Pascal) sous Linux Ubuntu, et utilise MySQL comme système de gestion de base de données.

Table des matières

1. Introduction
2. Étude de l'existant
3. Analyse des besoins
4. Conception
5. Réalisation
6. Tests et validation
7. Conclusion et perspectives
8. Annexes

1. Introduction

Le secteur agricole, en particulier les coopératives laitières, joue un rôle socio-économique important. Cependant, plusieurs coopératives manquent d'outils numériques adaptés à leurs besoins. Ce projet vise à fournir un logiciel simple, robuste et open source.

2. Étude de l'existant

Dans la majorité des coopératives laitières traditionnelles, la gestion des opérations repose encore sur des méthodes **manuelles ou semi-informatisées**, souvent à l'aide de cahiers, de tableaux Excel ou de registres papier. Cette gestion concerne plusieurs volets essentiels :

- **Adhésions** : L'enregistrement des nouveaux membres se fait manuellement, ce qui complique la mise à jour des informations et rend le suivi administratif peu fiable.
- **Livraisons quotidiennes** : Les membres livrent leur production de lait chaque jour. Les quantités sont notées manuellement, souvent sur des carnets ou des fiches, puis ressaisies plus tard dans un tableur ou un registre central. Ce processus est **chronophage et propice aux erreurs** (saisie, calculs, oublis...).
- **Paiements mensuels** : Les paiements dus aux adhérents sont calculés en fin de mois, généralement en fonction des volumes de lait livrés. L'absence d'un système automatisé entraîne souvent des **retards, des erreurs de calcul et des contestations**.
- **Suivi financier et analytique** : L'analyse de la rentabilité, la gestion des dettes, ou la planification des besoins en trésorerie sont très limitées, car il n'existe **aucun outil fiable de reporting** ou de visualisation des données.

Problèmes majeurs constatés :

1. **Risque élevé de perte de données** (en cas de sinistre, erreur humaine, ou simple négligence).
2. **Absence de traçabilité** : Difficile de retrouver l'historique d'un membre, de ses livraisons ou de ses paiements.
3. **Temps de traitement important** : La saisie et la vérification manuelles ralentissent considérablement les opérations.
4. **Pas de centralisation** : Chaque service (comptabilité, production, secrétariat) fonctionne en silo, sans accès direct aux données partagées.
5. **Difficulté de prise de décision** : Sans tableaux de bord ou indicateurs fiables, les dirigeants de la coopérative prennent des décisions sur la base de données incomplètes ou obsolètes.

3. Analyse des besoins

a) Fonctionnalités principales

Pour répondre efficacement aux problématiques identifiées dans l'étude de l'existant, le système à concevoir doit proposer les **fonctionnalités suivantes** :

1. Gestion des adhérents

- Enregistrement des nouveaux membres (nom, prénom, CIN, date d'adhésion, coordonnées, statut actif/inactif).
- Mise à jour des informations personnelles.
- Historique de participation et de livraison.

2. Suivi des livraisons quotidiennes

- Enregistrement des quantités de lait livrées par chaque adhérent, date par date.
- Saisie assistée avec contrôle de cohérence (quantité ≥ 0 , dates valides).
- Visualisation des livraisons par jour, semaine ou mois.

3. Calcul automatique des montants à payer

- Paramétrage du prix du litre de lait (modifiable selon les périodes).
- Calcul mensuel automatisé du montant dû à chaque adhérent en fonction du volume livré.
- Intégration de retenues, pénalités ou primes (facultatif).

4. Génération de factures et rapports

- Édition de bulletins de paiement mensuels individuels.
- Rapports synthétiques (livraisons par jour, paiements par mois, classement des adhérents...).
- Exportation des rapports en formats PDF/Excel pour impression ou archivage.

5. Accès sécurisé et gestion des utilisateurs

- Authentification avec nom d'utilisateur et mot de passe.
- Droits d'accès selon le profil de l'utilisateur (lecture, modification, suppression).

- Journalisation des opérations sensibles (sécurité et traçabilité).

6. Interface conviviale et adaptée

- Interface graphique simple, intuitive et optimisée pour une utilisation quotidienne.
 - Menus clairs, accès rapide aux fonctionnalités importantes.
 - Compatibilité avec les environnements Ubuntu (Linux).
-

b) Acteurs du système

1. Administrateur

- Rôle technique et organisationnel.
- Gère les utilisateurs, configure les paramètres système (prix du lait, accès, etc.).
- Supervise les rapports et les résultats globaux.

2. Employé de la coopérative

- Gère les adhérents (création, mise à jour).
- Enregistre les livraisons quotidiennes.
- Consulte les rapports de paiements et livre les bulletins aux membres.
- Peut avoir un accès restreint aux fonctions critiques du système.

4. Conception

4. Conception

La phase de conception vise à modéliser la structure du système d'information à travers des représentations claires et structurées. Elle comprend la **modélisation des données** (via un schéma de base de données) et la **modélisation fonctionnelle** (via un diagramme de cas d'utilisation).

a) Schéma de base de données

Un modèle relationnel a été élaboré pour répondre aux besoins identifiés. Il repose sur les tables principales suivantes :

1. *adherents*

- id : Clé primaire, identifiant unique.
- nom, prenom : Informations personnelles.
- date_adhesion : Date d'inscription dans la coopérative.
- cin : Numéro de la carte nationale d'identité.
- telephone, adresse : Coordonnées de contact.
- statut : Actif ou inactif.

2. *livraisons*

- id : Clé primaire.
- id_adherent : Clé étrangère vers la table adherents.
- date : Date de livraison du lait.
- quantite : Quantité de lait livrée (en litres).

3. *paiements*

- id : Clé primaire.
- id_adherent : Clé étrangère vers la table adherents.
- date_paiement : Date de paiement.
- montant : Montant payé à l'adhérent.

4. utilisateurs

- id : Clé primaire.
- nom_utilisateur, mot_de_passe : Informations d'authentification.
- role : Rôle de l'utilisateur (administrateur, employé).

Des clés étrangères assurent l'intégrité référentielle entre les tables, et des index sont prévus sur les colonnes fréquemment interrogées (dates, id_adherent...).

b) Diagramme de cas d'utilisation (UML)

Le système est centré sur deux types d'utilisateurs (acteurs) : **Administrateur** et **Employé**. Les principaux cas d'utilisation sont :

Acteur	Cas d'utilisation
Administrateur	- Gérer les utilisateurs- Gérer les adhérents- Configurer les paramètres (prix du litre)
Employé	- Enregistrer une livraison- Gérer les adhérents- Gérer les paiements- Générer les rapports

Description des cas principaux :

- **Gérer les adhérents** : Ajouter, modifier, supprimer ou consulter les membres.
- **Enregistrer une livraison** : Saisir la quantité de lait livrée pour un adhérent à une date donnée.
- **Gérer les paiements** : Calculer et enregistrer les montants dus, générer les bulletins de paie.
- **Générer les rapports** : Obtenir des synthèses par période, par adhérent, etc.
- **Gérer les utilisateurs** : Créer ou modifier les comptes et les droits d'accès.
- **Configurer les paramètres** : Modifier le prix du litre ou d'autres paramètres système.

Un diagramme UML visuel peut accompagner ce tableau, avec les acteurs reliés aux cas d'usage sous forme ovale

5. Réalisation (Implémentation)

La phase de réalisation consiste à concrétiser la conception par le **développement des interfaces et des fonctionnalités métiers**. Ce travail a été effectué sous **Lazarus** en environnement **Linux Ubuntu**, en utilisant une base de données **MySQL**.

a) Environnement de développement

Le choix de l'environnement technique s'est basé sur des critères de **performance, simplicité, compatibilité Linux**, et **liberté logicielle**.

- **IDE : Lazarus 2.2.x**, un environnement RAD (Rapid Application Development) open-source, similaire à Delphi.
 - **Langage : Free Pascal**, un langage compilé, stable et performant.
 - **Système d'exploitation : Ubuntu Linux**, pour sa fiabilité, son faible coût et sa compatibilité avec les outils open-source.
 - **Base de données : MySQL**, système de gestion de bases de données relationnelles très répandu, fiable et adapté à ce type d'application.
 - **Composants principaux :**
 - **ZeosLib** : Bibliothèque puissante pour la connexion aux bases SQL, utilisée avec ZConnection, ZQuery, etc.
 - Composants standard Lazarus : TEdit, TLabel, TButton, TDBGrid, etc.
-

b) Interface utilisateur

L'interface graphique a été conçue pour être simple, intuitive et fonctionnelle. Les formulaires ont été créés en mode glisser-déposer (RAD), ce qui a accéléré le développement.

Exemple : Formulaire de gestion des adhérents

- **Composants utilisés :**
 - TEdit : pour la saisie du nom, prénom, CIN.
 - TDateEdit : pour la date d'adhésion.
 - TDBGrid : affichage des adhérents avec pagination.

- TButton : pour les actions Ajouter, Modifier, Supprimer.
- ZConnection : gestion de la connexion MySQL.
- ZQuery : exécution des requêtes SQL.

Autres interfaces :

- **Livraisons** : formulaire avec calendrier et saisie rapide des quantités.
- **Paielements** : génération automatique des montants et affichage des paiements effectués.
- **Authentification** : formulaire sécurisé avec rôle utilisateur.

c) Pourquoi Lazarus/Free Pascal ? Comparaison avec C# et Java

Critère	Lazarus/Free Pascal	C# (.NET)	Java
Licence	Open source, gratuit, sans restriction	Licence propriétaire (Microsoft)	Open source mais Oracle peut poser des limites
Multiplateforme	Oui, Windows, Linux, macOS	Surtout Windows (Mono existe, mais limité)	Oui, mais nécessite JVM
Performance	Code natif, rapide, peu de dépendances	Interprété ou semi-compilé (CLR)	Moins rapide, dépend de la JVM
Consommation mémoire	Faible (code compilé en natif)	Moyenne à élevée	Élevée dans les interfaces riches
Simplicité de l'IDE	Lazarus est léger, facile à configurer	Visual Studio est très complet mais lourd	IDEs (NetBeans, IntelliJ) plus complexes
Déploiement Linux	Simple, natif, sans machine virtuelle	Moins simple (Mono, dépendances .NET Core)	Requiert JVM et configuration
Adapté aux petits projets	Excellente portabilité,	Suroptimisé pour des projets	Peut être surdimensionné

Critère	Lazarus/Free Pascal	C# (.NET)	Java
	léger, facile à maintenir	complexes	pour des besoins simples

Avantages spécifiques de Lazarus dans ce projet :

- **Interface RAD rapide à construire.**
- **Déploiement simplifié sous Linux Ubuntu**, utilisé dans les coopératives pour sa gratuité.
- **Accès direct aux bases de données via ZeosLib**, sans configuration lourde.
- **Pas de machine virtuelle nécessaire** (contrairement à Java).
- **Faible consommation de ressources**, idéal pour les PC modestes souvent utilisés dans les milieux ruraux

6. Tests et validation

Afin d'assurer la qualité du système et sa conformité aux exigences fonctionnelles, une série de **tests fonctionnels et techniques** ont été réalisés dans des conditions proches de l'utilisation réelle.

a) Objectifs des tests

- **Vérifier le bon fonctionnement de chaque module** : adhérents, livraisons, paiements.
- **Assurer la fiabilité des calculs** automatiques (totaux livrés, montants à payer).
- **Garantir la stabilité de la connexion avec la base de données MySQL sous Ubuntu.**
- **Détecter et corriger les éventuelles erreurs** avant mise en service.

b) Scénarios de tests effectués

Module testé	Scénario	Résultat attendu	Résultat obtenu
Gestion des adhérents	Insertion, modification, suppression, navigation dans le DBGrid	Données affichées et modifiables sans erreurs	Conforme
Connexion MySQL (ZeosLib)	Test de connexion, reconnexion après coupure, exécution de requêtes	Connexion stable, requêtes exécutées sans échec	Conforme
Saisie de livraisons	Enregistrement d'une livraison journalière, vérification du total	La quantité saisie s'ajoute au total mensuel	Conforme
Paiements et calculs	Saisie d'un paiement, calcul automatique du montant dû	Le reste à payer est mis à jour instantanément	Conforme
Authentification	Test de connexion	Accès	Conforme

Module testé	Scénario	Résultat attendu	Résultat obtenu
(sécurité)	avec mot de passe correct/incorrect	autorisé/refusé selon les cas	
Génération de rapports	Export PDF/Impression des états mensuels de livraisons et paiements	Rapports générés avec les bonnes données	Conforme

c) Méthodologie

- **Tests manuels** réalisés à chaque étape de développement.
 - **Scénarios réalistes** simulant les actions d'un utilisateur final (employé de la coopérative).
 - **Vérification croisée** avec les documents manuels de la coopérative pour comparer les résultats.
-

d) Résultat global

Tous les modules ont passé les tests fonctionnels avec succès. Le système s'est révélé :

- **Stable**
- **Fiable**
- **Conforme aux attentes exprimées lors de l'analyse des besoins**

Des tests complémentaires (de charge, de stress, de sécurité approfondie) pourraient être envisagés pour une future version déployée à grande échelle.

7. Conclusion et perspectives

Le projet a permis de fournir une solution complète à une coopérative laitière. À l'avenir, on pourrait :

- Ajouter une interface web ou mobile (avec Horse pour REST)
- Intégrer la gestion de stock et de matériel
- Générer des tableaux de bord graphiques

8. Annexes

- Captures d'écrans des interfaces
- Modèle E/A de la base de données
- Liste des composants utilisés
- Code source compressé